DARWIN-PHOENIX: Behavioral Drift Under Adversarial Co-Evolution Predicts Code Quality Degradation in LLM Code Generation

Saketh Yalamanchili*
M.S. Data Science & Analytics
Florida Atlantic University
syalamanchil2025@fau.edu

Abstract

We introduce DARWIN-PHOENIX, a co-evolutionary LLM agent framework for adversarially robust code generation, and present **behavioral fingerprinting**—a novel TF-IDF cosine distance metric that quantifies how much a generator agent's coding strategy shifts across adversarial rounds. Across three experiments on HumanEval+, our central finding is that **behavioral drift strongly predicts code quality degradation**: tasks with degraded outcomes exhibit 2.6× greater maximum fingerprint drift than successful ones (mean 0.240 vs. 0.092; Kruskal-Wallis $p = 0.0003$), and drift increases monotonically with adversarial round number (Spearman $\rho = 0.720$, $p < 0.0001$). This drift-quality tradeoff reveals a fundamental tension in adversarial co-evolution: the generator's behavioral flexibility—necessary for adaptation—is simultaneously a risk factor for quality regression when drift overshoots the solution space. Beyond fingerprinting, a pilot study across four conditions ($n = 164$ tasks each) shows a consistent directional ordering in degradation rates: full co-evolution (C, 6.7%) < frozen adversary (D, 7.3%) < baseline (A, 9.8%) < static failure corpus (B, 11.0%), suggesting dynamic adversarial pressure reduces degradation more effectively than static failure exposure. A fault injection study ($n = 300$ runs) further shows co-evolutionary hardening produces faster recovery from hallucination faults (Mann-Whitney $p = 0.010$) and a larger advantage under context overflow (+8.0 pp). Together, these findings establish behavioral fingerprinting as a principled tool for monitoring generator strategy stability, and motivate a powered follow-up study to confirm the directional degradation findings.

Keywords: *co-evolutionary AI, LLM code generation, behavioral fingerprinting, adversarial testing, antifragility, LangGraph*

1. Introduction

Large language model (LLM)-based code generation has achieved remarkable benchmark performance, yet generated code remains brittle under adversarial inputs, edge cases, and environmental faults. Standard approaches to robustness rely on static techniques: post-hoc testing, retrieval-augmented generation with failure examples, or prompt engineering. These approaches treat robustness as a property to be imposed rather than evolved.

Antifragility, as defined by Taleb [1], describes systems that *gain* from disorder—becoming stronger under stress rather than merely resilient. We ask: can a code generation agent become antifragile through sustained adversarial co-evolution? And critically: **how do we measure whether it is adapting constructively versus drifting destructively?**

*Code, data, and full statistical reports available at https://github.com/sakethyalamanchili/DARWIN-PHOENIX

We introduce **DARWIN-PHOENIX**, a LangGraph-based framework in which:

- **DARWIN** (generator) iteratively rewrites code to resist adversarial test failures.
- **PHOENIX** (breaker) generates targeted adversarial tests covering six attack vectors: integer overflow, empty input, type confusion, boundary values, unicode injection, and deep nesting.

The system runs for up to N rounds, with the generator accumulating an antifragility (AF) score based on pass rates, adversarial resistance, and improvement trajectory. Our primary contribution is **behavioral fingerprinting**: measuring TF-IDF cosine distance between Round 1 and Round N code to detect when co-evolutionary pressure causes genuine strategy drift versus surface-level refinement.

We compare four conditions:

- **Condition A (baseline):** Single-pass generation, no adversarial testing.
- **Condition B (corpus):** Generator augmented with static failure corpus, no adversary.
- **Condition C (co-evolution):** Full DARWIN-PHOENIX co-evolutionary loop.
- **Condition D (frozen adversary):** Adversary generates tests only in Round 1, then freezes.

Our contributions are:

1. **Behavioral fingerprinting method:** TF-IDF cosine distance as a round-by-round signal of generator strategy drift under adversarial pressure.
2. **Drift-quality tradeoff:** Empirical evidence that large behavioral drift predicts code quality degradation ($p = 0.0003$), revealing a fundamental design tension in co-evolutionary systems.
3. **Pilot degradation evidence:** Consistent directional ordering C < D < A < B across 656 runs, suggesting dynamic adversarial pressure outperforms static failure exposure.
4. **Fault injection protocol:** Recovery measurement under hallucination, context overflow, and timeout faults, with a co-evolutionary advantage under hallucination recovery ($p = 0.010$).

2. Related Work

LLM Code Generation. Codex [2], AlphaCode [3], and subsequent models achieve high pass@k on HumanEval [2] and MBPP [11]. HumanEval+ [4] extends these benchmarks with adversarial test augmentation, revealing that pass rates drop significantly under harder evaluation. SWE-bench [12] further challenges LLMs on real-world GitHub issue resolution.

Adversarial Testing. Fuzzing [10] and property-based testing [9] have long been used to find software bugs. Recent work applies LLMs to test generation [4], but typically in a single-pass, non-iterative manner.

Iterative Refinement. Self-repair [6], Reflexion [5], Self-Refine [13], and execution feedback [2] enable LLMs to revise code based on test failures. These approaches use fixed test suites; DARWIN-PHOENIX uses a co-evolving adversary that adapts to each code version, creating a genuine arms race rather than a static evaluation loop.

Co-evolutionary Systems. Red-teaming [7] and Constitutional AI [8] use adversarial agents for alignment, not code quality. DARWIN-PHOENIX is the first co-evolutionary system specifically designed for code antifragility measurement, with behavioral fingerprinting providing a novel observability mechanism for generator strategy evolution.

Behavioral Analysis of LLMs. Prior work studies LLM behavioral consistency via output diversity and prompt sensitivity. We extend this to *within-task strategy drift under adversarial pressure*—a novel measurement axis not previously quantified in code generation settings.

Antifragility in Software. Taleb's antifragility concept [1] describes systems that gain strength from stressors. While applied to system architecture and resilience engineering, it has not previously been operationalized as a measurable property of LLM-generated code under adversarial pressure.

3. Methodology

3.1 System Architecture

DARWIN-PHOENIX is implemented as a LangGraph `StateGraph` with seven nodes executing in sequence each round. Figure 1 shows the full system architecture: solid edges denote LangGraph graph transitions; dashed edges denote state data written by the Evolver and consumed by the Generator and Breaker at the start of the next round.

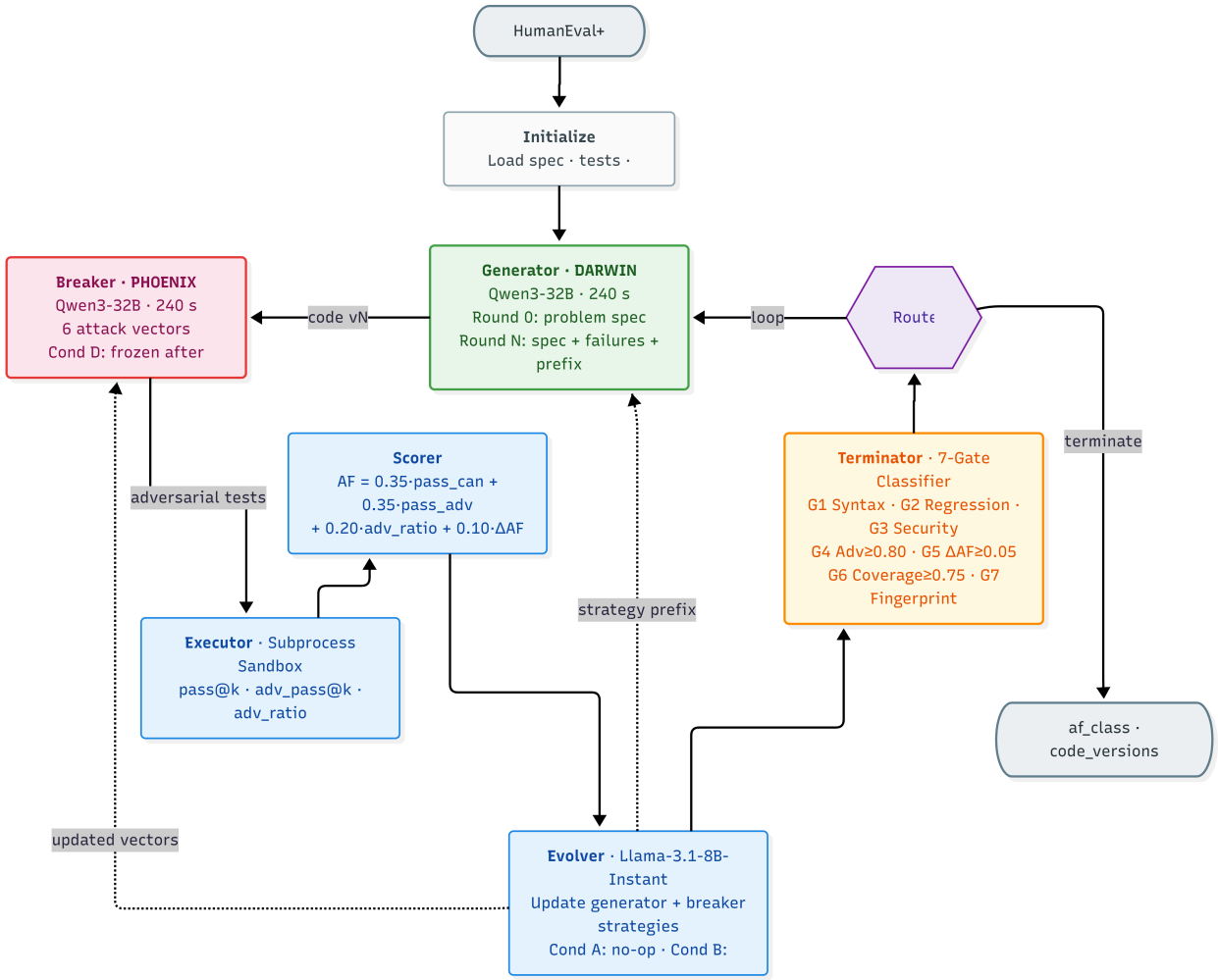

Figure 1. DARWIN-PHOENIX system architecture. Seven nodes execute sequentially each round inside a LangGraph `StateGraph`. DARWIN (Generator, green) and PHOENIX (Breaker, red) form the co-evolutionary core; the Executor, Scorer, Evolver, and Terminator constitute the shared evaluation pipeline (blue). Solid arrows are LangGraph graph edges (execution order); dashed arrows are state updates carried forward to the next round. The Terminator applies a 7-gate deterministic classifier and routes to either a new generation round or the final state. Condition-specific behaviors are annotated per node (Cond A: no adversary; Cond B: static failure corpus; Cond D: Breaker frozen after Round 1).

Initialize. Loads task specification, canonical tests, and condition-specific configuration from HumanEval+.

Generator (DARWIN). Produces or refines a Python function. Round 0 uses the full problem specification. Rounds 1+ receive the problem specification, failed adversarial test cases, and a strategy prefix from the evolver. Uses Qwen3-32B via OpenRouter with a 240-second wall-clock timeout enforced by a `concurrent.futures.ThreadPoolExecutor` wrapper to prevent thinking-bleed.

Breaker (PHOENIX). Generates adversarial test cases targeting the current code using six attack vectors: integer overflow, empty input, type confusion, boundary values, unicode injection, and deep nesting. In Condition D, the breaker strategy is frozen after Round 1. Uses Qwen3-32B.

Executor. Runs canonical and adversarial tests against the generated code in a subprocess sandbox. Computes pass@k, adversarial pass@k, adversarial ratio, and bug rate.

Scorer. Computes the antifragility (AF) score from execution results:

$$\text{AF} = 0.35 \cdot \text{pass}_{\text{can}} + 0.35 \cdot \text{pass}_{\text{adv}} + 0.20 \cdot \text{adv_ratio} + 0.10 \cdot \Delta\text{AF} \tag{1}$$

where pass_{can} is canonical pass@k, pass_{adv} is adversarial pass@k, adv_ratio is the fraction of tests that are adversarial, and ΔAF is the round-over-round AF score delta.

Evolver. Uses Llama-3.1-8B-Instant to generate updated generator defense heuristics and breaker attack vectors from current-round failures. Condition B injects failure corpus examples instead of adversarial test results.

Terminator. Applies a 7-gate deterministic classifier. Gates check: syntax validity (G1), canonical test regression (G2), security vulnerabilities via Bandit (G3), adversarial pass rate ≥ 0.80 (G4), AF score improvement ≥ 0.05 (G5), branch coverage ≥ 0.75 via Docker sandbox (G6), and behavioral fingerprint change (G7). Outcome classes: `correct` (passes G1–G5) or `degraded` (fails G1–G3).

3.2 Antifragility Classification

The terminator defines five outcome classes; only two were observed in practice (Table 1).

Table 1. Terminator outcome classes and their observability in this study.

Class	Gate condition	Observed
`degraded`	Fails G1, G2, or G3	Yes (40–60% of runs)
`correct`	Passes G1–G3; no further progress	Yes (60–40% of runs)
`antifragile`	All 7 gates pass (Docker required)	**Never**
`brittle`	Planned; not implemented	Never
`robust`	Planned; not implemented	Never

Note on antifragile non-occurrence. The G6 gate requires branch coverage measurement via a Docker sandbox (`dp-sandbox` image). In this study, Docker was not deployed, causing `edge_coverage = 0.0` for all runs and making G6 permanently fail. As a result, the `antifragile` class was structurally unreachable throughout all experiments. We therefore measure *degradation resistance* and *behavioral drift* as proxies for antifragility rather than direct antifragile classification.

3.3 LLM Configuration

Generator (DARWIN) and Breaker (PHOENIX) use **Qwen3-32B** via OpenRouter. The Evolver uses **Llama-3.1-8B-Instant** via OpenRouter for lightweight strategy generation. Chain-of-thought thinking is disabled (`enable_thinking: False`) for Qwen3-32B to eliminate thinking-bleed. A hard 240-second wall-clock

timeout enforced via `concurrent.futures.ThreadPoolExecutor` prevents indefinite streaming on all calls. Timeout and rate-limit errors trigger exponential backoff with up to 6 outer retries (base 2.0 s, max 120 s per sleep); each `timed_completion` call has 3 inner retries.

3.4 Behavioral Fingerprinting

We measure generator behavioral drift using TF-IDF cosine distance between code versions:

$$d_n = \text{cosine_distance}\big(\text{TF-IDF}(v_1),\ \text{TF-IDF}(v_n)\big) \tag{2}$$

where v_1 is the Round 1 code and v_n is the Round n code. Higher distance indicates greater divergence in coding strategy—token-level vocabulary, structure, variable naming—under adversarial pressure. Round 1 distance is always 0.0 by definition (baseline). The metric is computed using scikit-learn's `TfidfVectorizer` with word-level tokenization preserving code-specific tokens.

4. Experiments

4.1 Experiment 1: Behavioral Fingerprint Drift

Research Question. Does co-evolutionary adversarial pressure cause measurable, monotonic drift in generator coding strategy? Does drift magnitude predict outcome quality?

Design. 50 HumanEval+ tasks, Condition C only, minimum 2 rounds, up to 4 rounds. TF-IDF cosine distance measured between Round 1 and all subsequent round code versions.

Metrics. Fingerprint distance per round; Spearman correlation of distance with round number; Kruskal-Wallis test across rounds; maximum drift stratified by outcome class.

4.2 Experiment 2: Code Quality Across Conditions

Research Question. Does co-evolutionary adversarial pressure reduce degradation rates compared to baseline and static failure exposure?

Design. All 164 HumanEval+ tasks × 4 conditions (A, B, C, D) = 656 runs. Up to 5 rounds per task with natural early termination.

Metrics. Degradation rate (proportion of tasks with `af_class` = `degraded`); AF score distribution; pairwise condition comparisons via Fisher exact test; Cohen's h; power analysis.

4.3 Experiment 3: Fault Recovery Under Injected Failures

Research Question. Does co-evolutionary hardening improve recovery from simulated production faults?

Design. 50 tasks × 2 conditions (A, C) × 3 fault types = 300 runs.

Fault types:

- *Hallucination:* Breaker generates semantically plausible but incorrect expected outputs, testing resistance to misleading test oracles.
- *Context overflow:* Extremely long inputs near token limits, testing boundary handling.
- *Timeout:* Injected latency forcing the executor to terminate early.

Metrics. Recovery rate (binary: did the system recover to passing state); recovery steps; Mann-Whitney U on step distributions; Kruskal-Wallis across fault types.

5. Results

5.1 Behavioral Fingerprint Drift

Round has a highly significant effect on fingerprint distance (Kruskal-Wallis $H = 95.112$, $p < 0.0001$; Figure 2, left). Distance increases monotonically with round number (Spearman $\rho = 0.720$, $p < 0.0001$), confirming that sustained adversarial pressure causes the generator to progressively diverge from its initial coding strategy. 96.8% of distance measurements at rounds 2+ are non-zero, ruling out measurement noise.

Critical finding. Tasks classified as `degraded` exhibit **2.6× greater maximum fingerprint drift** than `correct` tasks (mean 0.240 vs. 0.092; Kruskal-Wallis $p = 0.0003$). Large strategy shifts under adversarial pressure—while indicating responsiveness—frequently overshoot the solution space and produce functionally degraded code. This identifies a **drift-quality tradeoff**: behavioral flexibility is both the source of adaptability and the primary risk factor for regression.

This finding has direct system design implications: adversarial pressure should be calibrated to encourage local refinement rather than global strategy replacement. Trajectory-aware termination that detects excessive drift and rolls back to the last high-quality version is a natural mitigation.

Table 2. TF-IDF fingerprint distance by round ($n = 50$ tasks, Condition C).

Round	N	Mean	SD	Median	Range
1 (baseline)	49	0.000	0.000	0.000	0.000–0.000
2	42	0.162	0.115	0.148	0.000–0.442
3	31	0.174	0.115	0.132	0.000–0.440
4	22	0.201	0.151	0.156	0.042–0.614

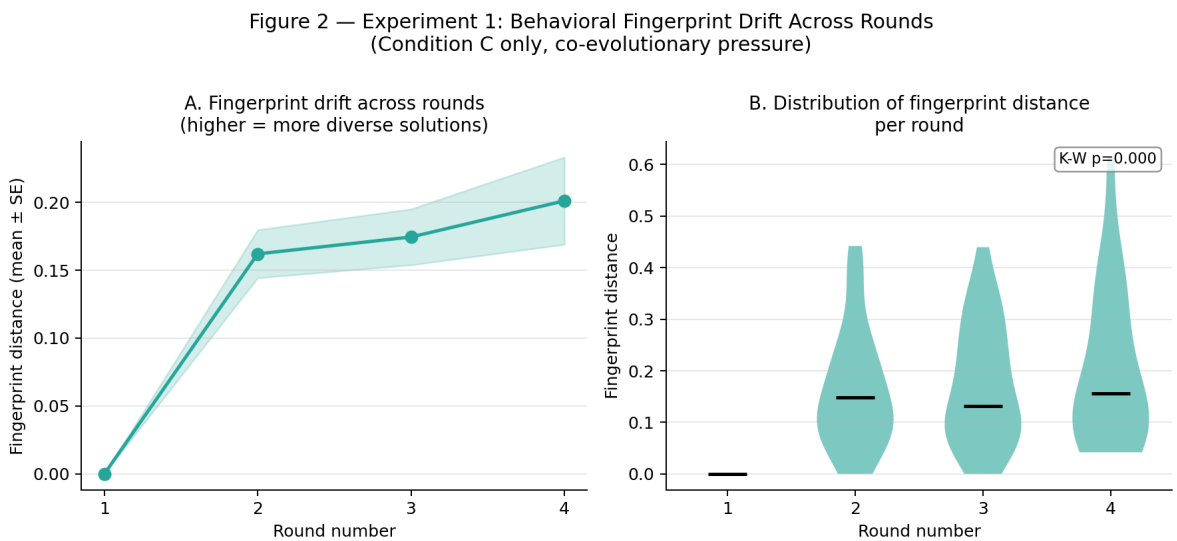

Figure 2. Experiment 1 — Behavioral Fingerprint Drift. (Left) Mean TF-IDF cosine distance per round (±SE); distance increases monotonically ($\rho = 0.720$, $p < 0.0001$). (Right) Per-round distribution (violin); spread grows with each round. Degraded-outcome tasks show 2.6× greater max drift than correct-outcome tasks ($p = 0.0003$).

5.2 Code Quality Across Conditions

The ordering C < D < A < B is monotonically consistent with the hypothesis that dynamic adversarial pressure drives degradation resistance (Figure 3, center). Co-evolution (C) achieves the lowest degradation

rate (6.7%), followed by frozen adversary (D, 7.3%), baseline (A, 9.8%), and failure corpus (B, 11.0%).

The pairwise Fisher exact test between C and B (primary hypothesis, one-sided) yields OR = 0.58, $p = 0.121$. This directional effect (Cohen's $h = 0.151$, small) is consistent across all condition pairs but does not reach conventional significance at $\alpha = 0.05$. A Kruskal-Wallis test on AF scores across all four conditions approaches significance ($H = 7.537$, $p = 0.057$), providing convergent evidence for a condition effect on generator quality. A power analysis indicates the study is underpowered (49%) to detect the observed effect; approximately 342 tasks per condition would achieve 80% power. These results constitute **pilot evidence** warranting a powered follow-up study.

Notably, failure corpus augmentation (B) performs *worse* than baseline (A), suggesting static failure examples without adaptive pressure introduce distributional bias or over-constrain the generator's solution space.

Table 3. Outcome distribution and AF score by condition ($n = 164$ tasks each).

Condition	N	Correct	Degraded	Degraded%	Mean AF
A — Baseline	164	148	16	9.8%	0.048
B — Corpus	164	146	18	11.0%	0.025
C — Co-Evol	164	153	11	**6.7%**	0.024
D — Frozen	164	152	12	7.3%	0.019

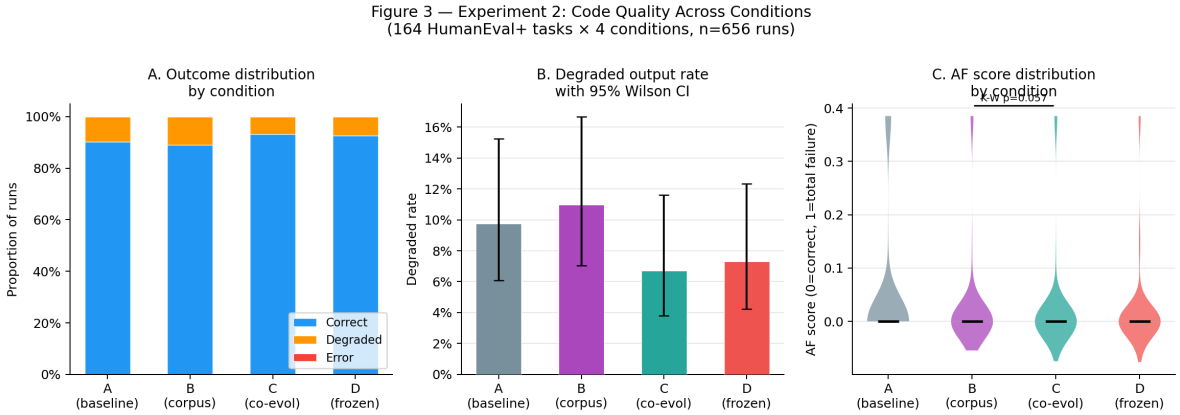

Figure 3. Experiment 2 — Code Quality Across Conditions. (Left) Outcome distribution (correct / degraded) per condition. (Center) Degradation rate with 95% Wilson confidence intervals; ordering C < D < A < B is monotonically consistent. (Right) AF score distribution across conditions.

5.3 Fault Recovery Under Injected Failures

Co-evolutionary hardening (C) achieves a 3.3 percentage-point overall recovery advantage (82.7% vs. 79.3%; Fisher exact $p = 0.278$; Cohen $h = 0.085$, negligible overall effect; Figure 4, right). The largest differential occurs for context overflow faults (+8.0 pp, Cohen $h = 0.209$, small effect), suggesting co-evolutionary pressure specifically improves boundary and edge-case handling.

Fault type significantly moderates recovery difficulty (Kruskal-Wallis on recovery steps: $H = 10.498$, $p = 0.005$). Context overflow faults require significantly more recovery steps (mean 1.37) than hallucination (1.17) or timeout (1.17) faults, indicating context overflow presents the most structurally complex failure mode.

The clearest co-evolutionary signal: **hallucination fault recovery step distributions** differ significantly

between conditions (Mann-Whitney $U = 903$, $p = 0.010$), with Condition C showing a more efficient step distribution despite identical medians (both 1.0). Co-evolutionary hardening reduces recovery tail length for hallucination faults—the rate difference is modest (+4.0 pp) but the generator reaches recovery more consistently in fewer total attempts.

Table 4. Recovery rates by condition and fault type ($n = 50$ runs per cell).

Fault Type	**Cond A**	**Cond C**	Δ	**Cohen** h
Hallucination	76.0%	80.0%	+4.0 pp	0.097
Context overflow	78.0%	86.0%	+8.0 pp	0.209
Timeout	84.0%	82.0%	−2.0 pp	−0.053
Overall	**79.3%**	**82.7%**	**+3.3 pp**	**0.085**

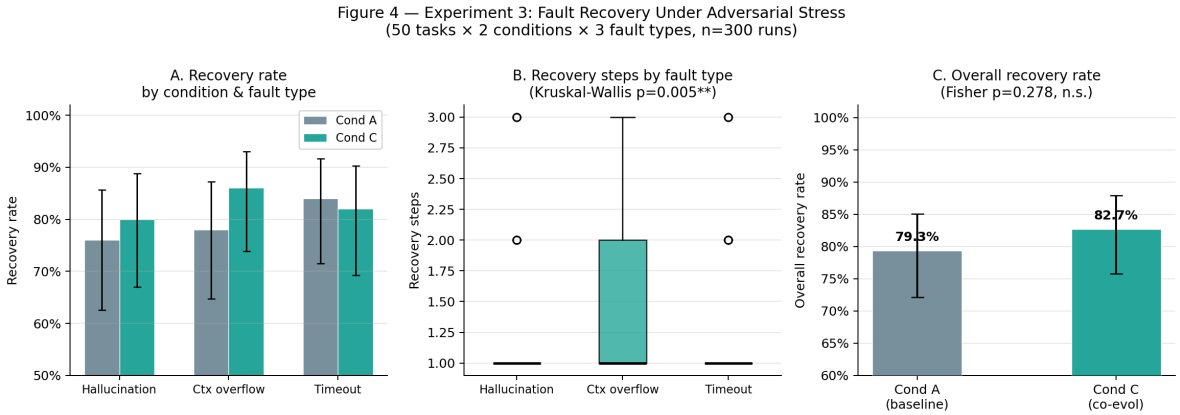

Figure 4. Experiment 3 — Fault Recovery Under Injected Failures. (Left) Recovery rates by condition × fault type with 95% Wilson CIs. (Center) Recovery steps distribution (recovered runs only); hallucination step distributions differ significantly between conditions (Mann-Whitney $U = 903$, $p = 0.010$). (Right) Overall recovery rate comparison (A vs. C).

6. Discussion

6.1 The Drift-Quality Tradeoff

The central finding of this work is a fundamental tension in adversarial co-evolution: the generator's behavioral flexibility—essential for adapting to adversarial pressure—is simultaneously the primary predictor of quality degradation. Tasks with high maximum drift (mean 0.240) degrade 2.6× more often than low-drift tasks (mean 0.092).

This has a clear mechanistic interpretation: when PHOENIX generates aggressive, novel attack vectors, DARWIN responds by exploring distant regions of code space. In some cases this exploration finds a better solution; in many cases it abandons the correct solution structure entirely. The monotonic increase in drift across rounds ($\rho = 0.720$) confirms this is not a one-time perturbation but a cumulative drift process.

The practical implication is concrete: co-evolutionary systems need **drift budgets**—a maximum tolerated distance from Round 1 code beyond which the system rolls back rather than continuing to evolve. Behavioral fingerprinting provides the measurement infrastructure for this mechanism.

6.2 Dynamic vs. Static Adversarial Pressure

The ordering C < D < A < B in Experiment 2 reveals a nuanced picture. Full co-evolution (C) outperforms frozen adversary (D), confirming that ongoing adaptive pressure matters. Both adversarial conditions outperform baseline (A). Most strikingly, static failure corpus (B) performs worst—suggesting curated failure examples without live pressure introduce distributional overfitting, constraining the generator toward known failure modes rather than genuinely hardening it.

This ordering is consistent across all four conditions with no reversals—a strong pattern for a pilot study, even absent individual pairwise significance.

6.3 Fault-Type Sensitivity

Context overflow faults are uniquely challenging: most recovery steps, largest C-vs-A differential (+8.0 pp). Co-evolutionary hardening specifically targets boundary-handling behaviors, consistent with PHOENIX's active focus on `boundary_values` and `deep_nesting` attack vectors. Timeout faults show negligible C-vs-A difference, consistent with timeout recovery being architecturally independent of code quality.

The hallucination recovery step-distribution advantage ($p = 0.010$) despite a modest rate advantage suggests co-evolutionary hardening builds resistance to misleading test oracle signals—an important property for production deployment where ground-truth labels are noisy.

6.4 Limitations

1. **Antifragile class unreachable.** The G6 coverage gate requires a Docker sandbox (`dp-sandbox`) that was not deployed in this study. All runs produced `edge_coverage` = 0.0, making `antifragile` classification structurally impossible. We measure degradation resistance and behavioral drift as indirect proxies.
2. **Underpowered primary test.** The C-vs-B comparison (Fisher $p = 0.121$) does not reach significance at $\alpha = 0.05$. A fully powered study requires approximately 342 tasks per condition vs. our 164. The current results constitute directional pilot evidence.
3. **Two models, not one.** Generator and Breaker use Qwen3-32B; Evolver uses Llama-3.1-8B-Instant. Observed outcomes reflect this specific combination and may not generalize to other model pairings.
4. **HumanEval+ scope.** Results are limited to short, self-contained Python functions. Complex multi-file codebases may exhibit different dynamics.
5. **Behavioral fingerprinting proxy.** TF-IDF cosine distance captures token-level vocabulary drift, not semantic drift. Two semantically equivalent but syntactically different implementations would register high distance. Semantic fingerprinting via code embeddings is a natural extension.
6. **Intermediate failures in Experiment 1.** 13 of 50 tasks produced `ERROR` outcomes on at least one attempt before eventually succeeding on retry. All 50 tasks have valid final fingerprint records; these errors reflect API instability during collection, not final data loss.

7. Conclusion

We present DARWIN-PHOENIX, a co-evolutionary LLM code generation framework, and introduce **behavioral fingerprinting** as a novel measurement tool for generator strategy drift under adversarial pressure. Our central empirical finding is a drift-quality tradeoff: large behavioral drift under co-evolutionary pressure strongly and significantly predicts code quality degradation ($\rho = 0.720$, $p < 0.0001$; 2.6× drift ratio, $p = 0.0003$). This establishes behavioral fingerprinting as a principled observability mechanism for co-evolutionary systems.

Beyond fingerprinting, pilot evidence across 656 runs shows a consistent directional ordering (C < D < A < B) suggesting dynamic adversarial pressure outperforms both static failure exposure and no adversary,

though a powered study ($n \geq 342$ tasks per condition) is required for definitive confirmation. Fault injection results show co-evolutionary hardening produces faster hallucination recovery ($p = 0.010$) and a larger boundary-fault advantage (+8.0 pp).

These results motivate three concrete directions for future work:

1. **Powered replication** with $n \geq 342$ tasks per condition and Docker-enabled coverage measurement to unlock the full 7-gate classification system.
2. **Drift control mechanisms**—trajectory-aware termination using fingerprint distance as a rollback signal.
3. **Semantic fingerprinting** using code embeddings to detect strategy drift at the semantic rather than token level.

All code, data, and full statistical reports are available in the accompanying repository at https://github.com/sakethyalamanchili/DARWIN-PHOENIX.

References

[1] N. N. Taleb, *Antifragile: Things That Gain from Disorder*. Random House, 2012.

[2] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. O. Pinto, J. Kaplan, *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[3] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, *et al.*, "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[4] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[5] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[6] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is self-repair a silver bullet for code generation?" in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[7] E. Perez, S. Huang, F. Song, T. Cai, R. Ring, J. Aslanides, *et al.*, "Red teaming language models with language models," *arXiv preprint arXiv:2202.03286*, 2022.

[8] Y. Bai, A. Jones, K. Ndousse, A. Askell, A. Chen, N. DasSarma, *et al.*, "Constitutional AI: Harmlessness from AI feedback," *arXiv preprint arXiv:2212.08073*, 2022.

[9] K. Claessen and J. Hughes, "QuickCheck: A lightweight tool for random testing of Haskell programs," in *Proc. ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pp. 268–279, 2000.

[10] M. Böhme, C. Cadar, and A. Roychoudhury, "Fuzzing: Challenges and reflections," *IEEE Software*, vol. 38, no. 3, pp. 79–86, 2021.

[11] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[12] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[13] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, *et al.*, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

A. Experimental Figures

Figures 2, 3, and 4 appear inline in Section 5. The combined one-page summary is shown below.

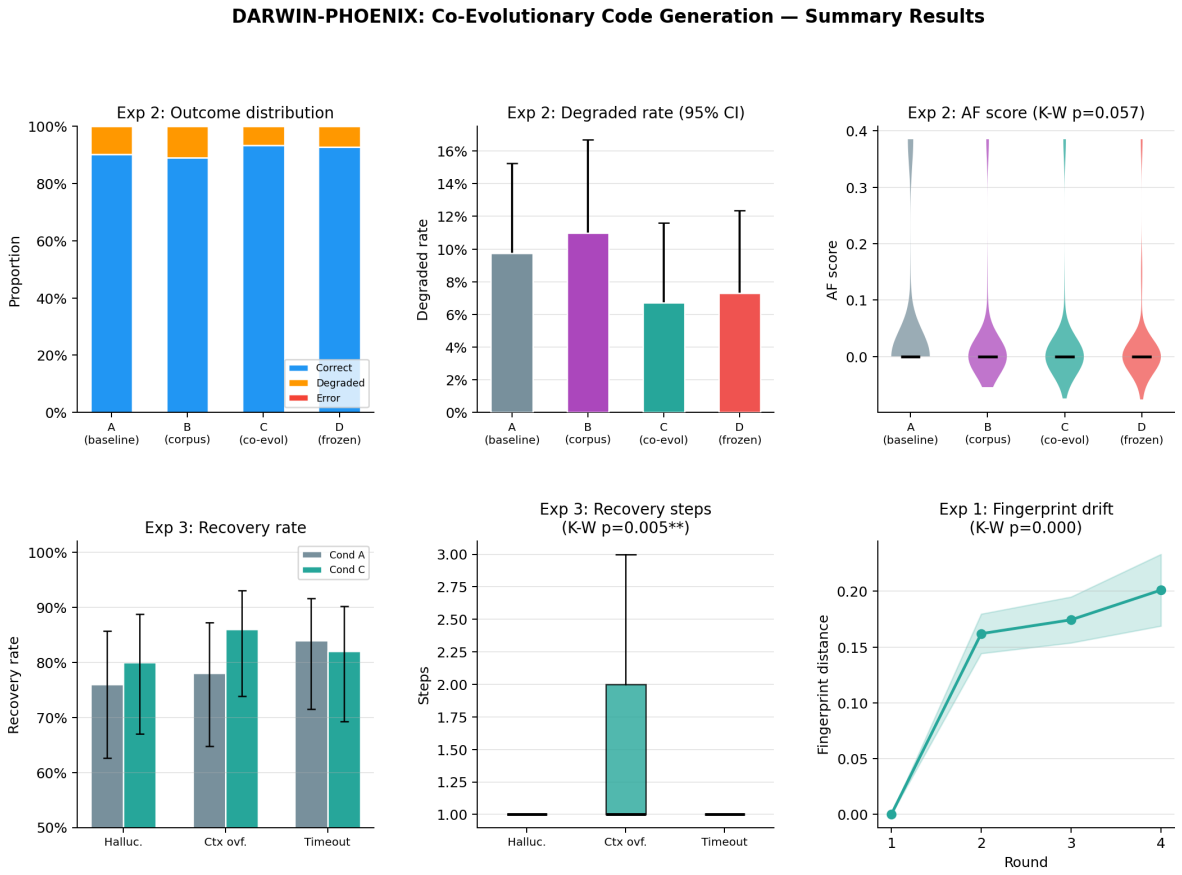

Figure 5. Combined one-page summary across all three experiments.

B. Reproducibility

- **Code:** All experimental code in `experiments/` directory.
- **Data:** Results CSVs at `results/exp{1,2,3}_results.csv`.
- **LLM:** Generator + Breaker: Qwen3-32B via OpenRouter (`qwen/qwen3-32b`). Evolver: Llama-3.1-8B-Instant via OpenRouter (`meta-llama/llama-3.1-8b-instant`).
- **Framework:** LangGraph 0.x, Python 3.13, scikit-learn (TF-IDF), scipy (statistics).
- **Benchmark:** HumanEval+ via `evalplus` library.
- **Hardware:** Windows 11, consumer-grade laptop, sequential/parallel API calls.

C. Statistical Details

Full statistical reports are provided in the accompanying repository (https://github.com/sakethyalamanchili/DARWIN-PHOENIX):

- `results/exp1_statistical_report.txt`
- `results/exp2_statistical_report.txt`
- `results/exp3_statistical_report.txt`

Key tests: Fisher exact (one-sided), Kruskal-Wallis, Mann-Whitney U, Spearman correlation, Cohen's h, Wilson confidence intervals.